

Summary of Page Replacement Algorithms

1 Core Idea

Review

Page replacement algorithms choose which resident page to evict when a page fault occurs and no free frame is available.

The best algorithm would evict a page that will not be needed soon. Real systems must approximate this using available information such as arrival order, R bits, M bits, counters, and working-set age.

2 Main Comparison

Algorithm	Comment
Optimal	Not implementable, but useful as a benchmark.
NRU	Very crude approximation of LRU.
FIFO	Simple, but might throw out important pages.
Second chance	Big improvement over FIFO.
Clock	Realistic implementation of second chance.
LRU	Excellent, but difficult to implement exactly.
NFU	Fairly crude approximation to LRU.
Aging	Efficient algorithm that approximates LRU well.
Working set	Good idea, but somewhat expensive to implement.
WSClock	Good efficient practical algorithm.

3 Algorithm Families

Family	Algorithms	Main idea
Future-based	Optimal	Use exact future references. Best possible, but impossible in real systems.
Bit-based	NRU	Use R and M bits to classify pages.
Queue-based	FIFO, second chance, clock	Track arrival order, then improve it using the R bit.
History-based	LRU, NFU, aging	Use past references to estimate future usefulness.
Working-set-based	Working set, WSClock	Keep active pages in memory and evict pages outside the working set.

4 Short Explanations

4.1 Optimal

Optimal

Evicts the page whose next use is farthest in the future.

It gives the lowest possible number of page faults for a known reference string. It cannot be used in practice because the operating system does not know the future.

4.2 NRU

NRU

Divides pages into four classes using R and M , then chooses randomly from the lowest-numbered nonempty class.

It is simple, but crude. It only has coarse information about whether pages were referenced or modified.

4.3 FIFO

FIFO

Evicts the page that has been in memory the longest.

It is easy to implement using a queue, but age in memory is not the same as usefulness. An old page may still be heavily used.

4.4 Second Chance and Clock

Second Chance / Clock

Second chance improves FIFO by checking the R bit. Clock implements the same idea efficiently using a circular list.

If $R = 1$, the page is spared once and R is cleared. If $R = 0$, it can be evicted.

4.5 LRU

LRU

Evicts the page that has not been used for the longest time.

LRU is a strong approximation to optimal, but exact LRU requires costly per-reference tracking or special hardware.

4.6 NFU and Aging

NFU / Aging

NFU counts references. Aging improves NFU by shifting counters so old references fade.

NFU does not forget old activity, so it can preserve pages that are no longer useful. Aging is much better because recent references matter more than old ones.

4.7 Working Set and WSClock

Working Set / WSClock

The working set is the group of pages a process is actively using. WSClock combines this idea with the clock algorithm.

The working set algorithm reduces page faults and thrashing, but scanning can be expensive. WSClock keeps good performance while being efficient to implement.

5 Best Practical Choices

Most Important Practical Algorithms

Two especially important practical algorithms are:

- **Aging**: based on LRU history.
- **WSClock**: based on working-set behavior.

Both give good paging performance and can be implemented efficiently.

6 Final Summary

One-Box Summary

Optimal is best theoretically, but impossible in practice.
FIFO is simple, but may evict useful pages.
Clock is a practical improvement over FIFO/second chance.
Aging approximates LRU efficiently.
WSClock combines working-set ideas with clock efficiency.